

Special Need Vehicle Rental

User Flow Guide & Hosting Estimate

Version: 1.1.0 **Last Updated:** June 2026 (updated to reflect Stripe payment integration) **Jurisdiction:** New South Wales, Australia **Audience:** Product owner, UX designers, developers, operations staff

Table of Contents

- 1. [Overview](#)
- 2. [Renter User Flow — Step-by-Step](#)
- 3. [Admin / Staff User Flow](#)
- 4. [Edge Cases & Error Flows](#)
- 5. [Cancellation & Refund Flow](#)
- 6. [Accessibility & Inclusive Design](#)
- 7. [Screen-by-Screen UI Guide](#)
- 8. [Mobile & Responsive Design](#)
- 9. [Email Notification Flows](#)
- 10. [Cloud Hosting Cost Estimate](#)
- 11. [Deployment Checklist](#)
- 12. [Operational Launch Checklist](#)
- 13. [Payment Testing Guide \(Stripe\)](#)

1. Overview

This guide describes every step a user takes — from arriving at the application from the facility website, through searching for vehicles, completing a booking, and receiving confirmation — as well as the parallel flows for facility staff using the admin dashboard.

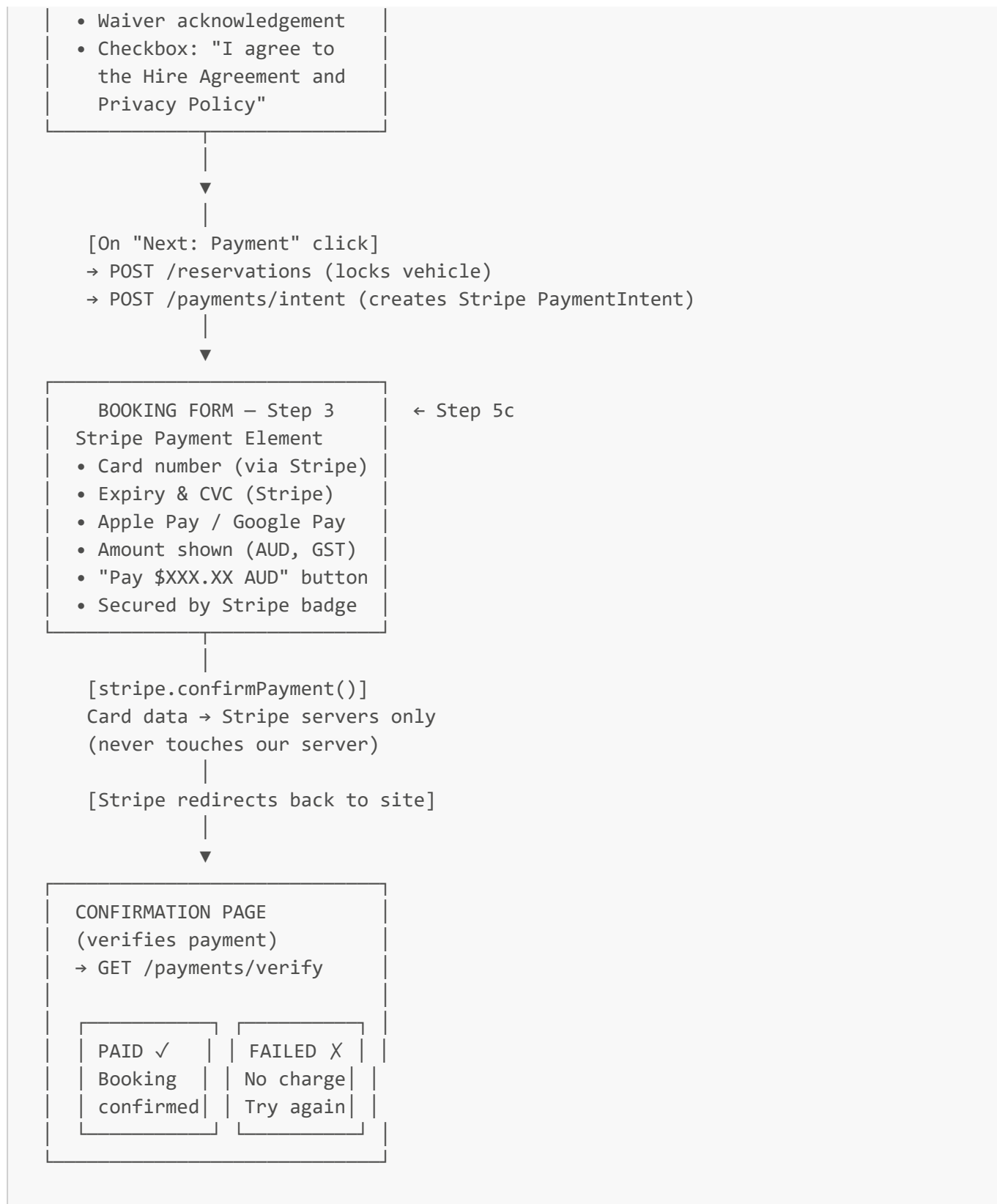
The application supports two primary user types:

User Type	Description	Access Level
Renter (Customer)	Member of the public, family member, carer or organisation booking a vehicle	Public — no login required
Admin (Staff)	Facility staff managing vehicles, bookings, and operations	Private — password protected

2. Renter User Flow — Step-by-Step

2.1 Complete Renter Journey





2.2 Step-by-Step Detail

Step 1 — Landing Page

User arrives from facility website link.

The landing page must establish trust immediately. Content includes:

- Facility name and logo prominently displayed
- Short tagline: "Our vehicles, available to you — safe and accessible hire for our community"
- 2–3 sentences explaining the service: the facility's own fleet is available for hire when not in use for resident transport, making it affordable and accessible for the local community
- Key trust signals: "Operated by [Facility Name] — trusted in our community since [Year]. These are our own vehicles — the same ones our residents rely on."
- Prominent "**Check Availability**" button (primary CTA)
- Secondary: "**Call us to book**" phone number (accessibility alternative)
- Brief vehicle highlights (3–4 photos or icons: sedan, wagon, wheelchair van)
- Footer: links to Privacy Policy, Terms & Conditions, Contact, and NSW Fair Trading notice

NSW compliance note: A link to the facility's complaints procedure and NSW Fair Trading must be accessible from the footer.

Operational note for staff: Vehicles listed in the system are facility-owned. Before any vehicle is made available for public hire, confirm it is not scheduled for resident transport use during that window. Facility resident transport always takes priority over public bookings.

Step 2 — Availability Search

User selects their required date/time range.

UI Element	Behaviour
Start Date/Time picker	Date + time in 30-minute increments; minimum = now + 2 hours
End Date/Time picker	Must be after start; maximum 7 days ahead of start
Timezone display	"All times shown in Sydney time (AEST/AEDT)" — always visible
Vehicle type filter	Optional dropdown: All / Sedan / Wagon / Van / Wheelchair-Accessible
Search button	Triggers API call; shows loading indicator

Validation (client + server):

- Start must be at least 2 hours from now (allows staff to prepare)
- End must be after start by at least 1 hour
- Dates must not be in the past
- Error messages displayed inline, associated with relevant fields

Step 3 — Vehicle Results List

System displays available vehicles for the requested window.

Each vehicle card displays:

- Vehicle photo (or placeholder icon)
- Vehicle name and type
- Accessibility features summary (e.g., "Wheelchair ramp, 2 anchor points")

- Seating capacity
- Price for the requested duration (AUD, GST-inclusive)
- Hourly/daily rate breakdown
- "Book This Vehicle" button

If no vehicles are available:

- Message: *"No vehicles available for your selected time. Try a different date or time."*
- Quick-select suggestions: "Try tomorrow" / "Extend by 30 minutes" buttons

Step 4 — Vehicle Detail Page

User reviews full vehicle information before committing.

Sections:

- Photo gallery (swipeable on mobile)
- Full accessibility notes (ramp details, wheelchair dimensions, hand controls)
- Add-on services with pricing
- Price breakdown table:

Duration rental:	\$88.00
Accessibility premium:	\$11.00
Add-ons:	\$5.50
Subtotal (excl. GST):	\$95.00
GST (10%):	\$9.50
Total:	\$104.50 AUD

- Pickup location and instructions
- Cancellation policy summary (e.g., "Free cancellation up to 24 hours before pickup")
- "Proceed to Booking" primary button

Step 5a — Personal Details (Booking Form, Step 1 of 3)

User enters their contact information.

Field	Type	Validation
Full name	Text	Required; 2–100 characters
Email address	Email	Required; valid email format
Mobile phone	Tel	Required; Australian format (+61 or 04xx xxx xxx)
Intended use	Textarea	Optional; placeholder: "e.g., medical appointment, community outing"
Progress indicator	Visual	Step 1 of 3 shown

Step 5b — Terms & Conditions (Booking Form, Step 2 of 3)

User reads and accepts hire agreement.

- Full terms and conditions displayed in a scrollable region
 - Waiver text summarising renter obligations (care of vehicle, valid licence required, fuel return policy)
 - Privacy Policy link (opens in new tab)
 - Checkbox: **"I have read and agree to the Vehicle Hire Agreement, Terms & Conditions, and Privacy Policy"**
 - Checkbox is required — booking cannot proceed without acceptance
 - Timestamp of acceptance is stored in the database
 - **NSW compliance:** Under the Electronic Transactions Act 2000 (NSW), this checkbox constitutes a valid electronic agreement. The confirmation email provides the accepted terms for the customer's records.
-

Step 5c — Payment (Booking Form, Step 3 of 3)

User pays via Stripe — card data never reaches the application server.

When the user clicks "Next: Payment" at the end of Step 2, two things happen automatically in the background:

1. The reservation is created in the database (the vehicle is locked for this customer)
2. A Stripe PaymentIntent is created and the Stripe Payment Element loads

The user sees:

UI Element	Behaviour
Amount summary bar	Total (AUD, GST-inclusive) and GST component shown — read-only
Stripe Payment Element	Stripe's hosted card input — renders credit/debit card fields, and Apple Pay / Google Pay if the browser supports them
"Pay \$X.XX AUD" button	Disabled until Stripe Element is ready and loaded
Security badge	"Secured by Stripe. Your card details are never stored on our servers."
Back button	Cancels the reservation and returns to Step 2 so the vehicle is freed up

What happens when the user clicks Pay:

1. `stripe.confirmPayment()` is called — card details go directly from the browser to Stripe's servers
2. Stripe processes the payment
3. On success: Stripe redirects the browser to the Confirmation page with a `payment_intent` parameter in the URL
4. On failure: an error message appears inline (e.g. "Your card was declined. Please try a different card.")

Security: The application server never receives or stores card numbers, CVCs, or full expiry dates at any point. The only thing stored in the database is the Stripe `PaymentIntent` ID (e.g. `pi_3abc123...`) in the `payment_token` column. This keeps the application fully out of PCI scope.

Confirmation Page

Payment verified — booking confirmed.

After Stripe redirects back to the site, the confirmation page calls the backend to verify the payment status with Stripe. The page displays differently depending on the payment outcome:

Payment succeeded (normal case):

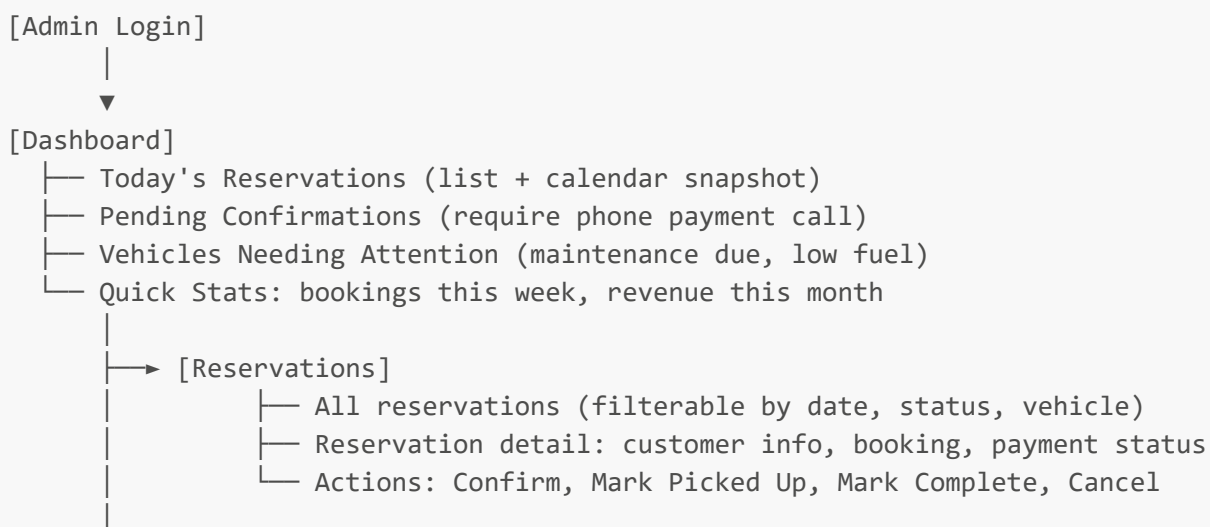
- Green checkmark icon and "Booking Confirmed!" heading
- "Payment received" success banner
- Booking reference number (e.g., `SNVR-20260615-042`)
- Booking summary: vehicle, dates, total paid (AUD, GST breakdown)
- Payment status: **Paid** (shown in green)
- Pickup address and instructions
- Next steps: bring driver's licence, sign hire agreement on arrival
- Cancel booking link (for change of plans)
- "Return to Home" button

Payment failed:

- Red X icon and "Payment Not Completed" heading
 - "No charge has been made" confirmation
 - "Try Again" button → returns to availability search
 - Staff phone number for manual assistance
-

3. Admin / Staff User Flow

3.1 Admin Dashboard Overview





3.2 Daily Admin Workflow

Morning (start of shift):

1. Log in to admin dashboard
2. Review "Today's Reservations" — note pickup times and vehicles
3. Check "Pending Confirmations" — call customers to confirm payment
4. Ensure vehicles scheduled for pickup today are cleaned, fuelled, and inspected
5. Mark vehicles in maintenance as unavailable in system if not already done

During shift: 6. When customer arrives: verify ID and driver's licence 7. Complete vehicle walk-around with customer; take photos of existing damage 8. Obtain signature on physical hire agreement 9. Hand over keys; update reservation status to "Picked Up" in system 10. When vehicle returned: inspect vehicle; photograph; mark reservation "Completed"

End of shift: 11. Check tomorrow's bookings; communicate any issues to next shift 12. Log any incidents or damage in the system notes

3.3 Vehicle Management Workflow

Adding a new vehicle:

1. Navigate to Vehicles → Add Vehicle
2. Enter: name, type, plate, capacity, hourly/daily rates, accessibility notes
3. Upload photos (recommended: front, rear, interior, ramp if applicable)
4. Set buffer time (minutes between bookings for cleaning)
5. Set status to "active" — vehicle is now bookable
6. Save

Setting a vehicle to maintenance:

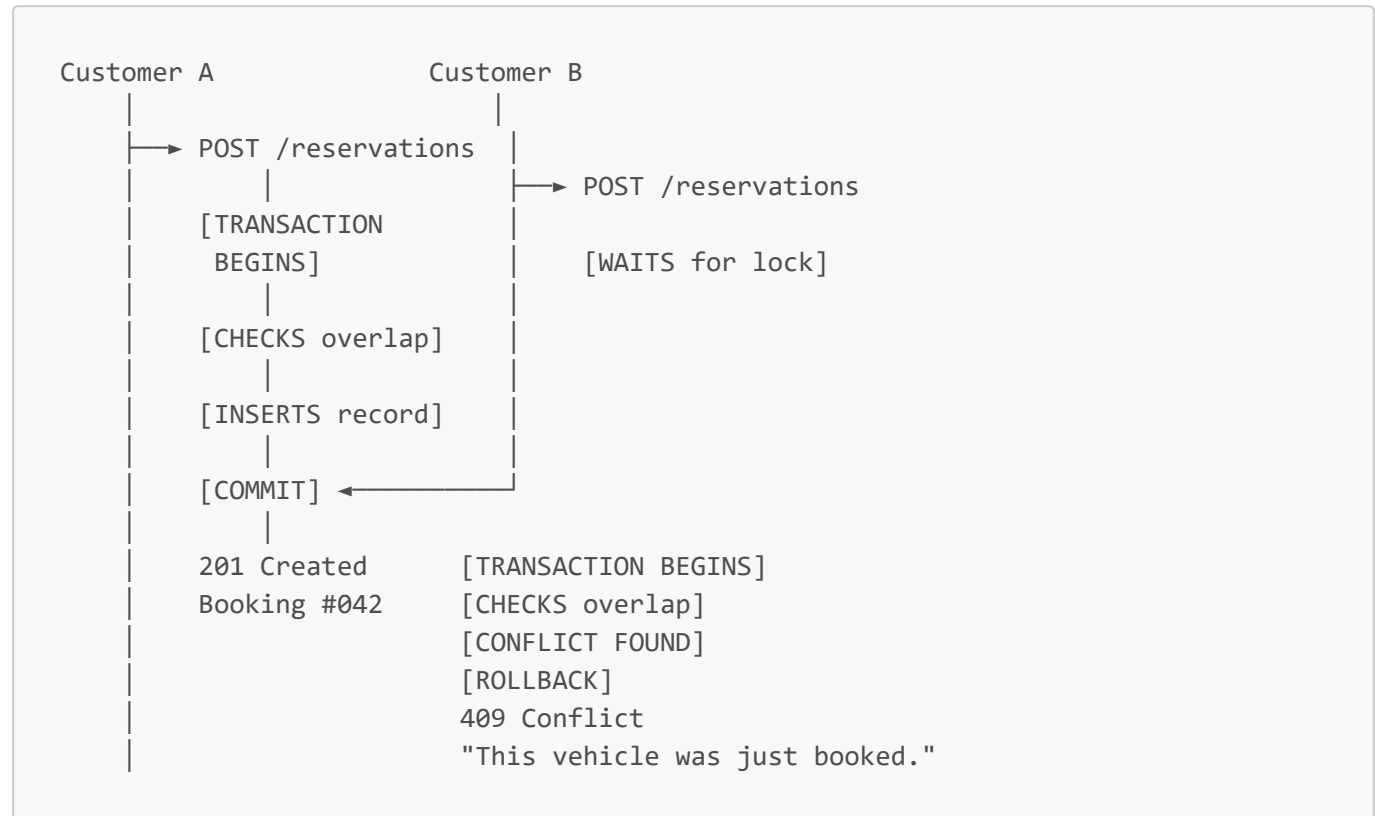
1. Navigate to Vehicles → select vehicle
2. Set status to "maintenance"
3. Set "maintenance until" date/time
4. System automatically blocks new bookings during this period

- Existing bookings in the maintenance window are flagged for manual review

4. Edge Cases & Error Flows

4.1 Concurrent Booking Conflict

Scenario: Two customers submit a booking for the same vehicle and time slot simultaneously.



User experience for Customer B:

- Friendly error message: *"We're sorry — this vehicle was just booked by another customer. Please choose a different vehicle or time."*
- Suggestions displayed: "Other available vehicles for your time" / "Next available slot for this vehicle"
- Customer is NOT charged — the conflict is detected before the Stripe PaymentIntent is created, so no payment session is opened for the failed attempt

Note on timing: With the Stripe integration, the reservation is created (and the vehicle locked) when the user clicks "Next: Payment" at the end of Step 2 — before the card form appears. This means the conflict is detected earlier than in Phase 1: rather than at the final submit, it's caught before the customer sees the payment form at all.

4.2 Validation Errors

All form errors appear:

- Inline beneath the relevant field (red border + error text)
- In a summary at the top of the form for screen reader accessibility (ARIA live region)
- Specific, actionable messages — not generic "Error"

Error	Message Shown
Invalid email	"Please enter a valid email address (e.g., name@example.com)"
Phone format	"Please enter a valid Australian mobile number (e.g., 0412 345 678)"
Date in past	"Please select a start time at least 2 hours from now"
End before start	"Return time must be after pickup time"
Terms not accepted	"You must agree to the hire terms and conditions to proceed"

4.3 Session / Timeout

If the user takes more than 30 minutes to complete the booking form (the vehicle is held speculatively for 15 minutes after Step 4):

- System shows a warning at 10 minutes remaining: *"Your vehicle hold expires in 10 minutes. Please complete your booking."*
- If time expires: form clears; user is returned to Step 2 with message: *"Your vehicle hold has expired. Please search again."*

5. Cancellation & Refund Flow

5.1 Customer Self-Cancellation

Cancellation link is included in every confirmation email.

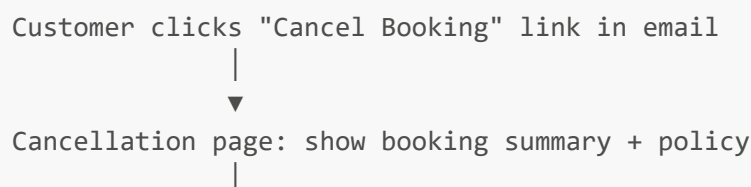
Cancellation policy (recommended; adjust as required):

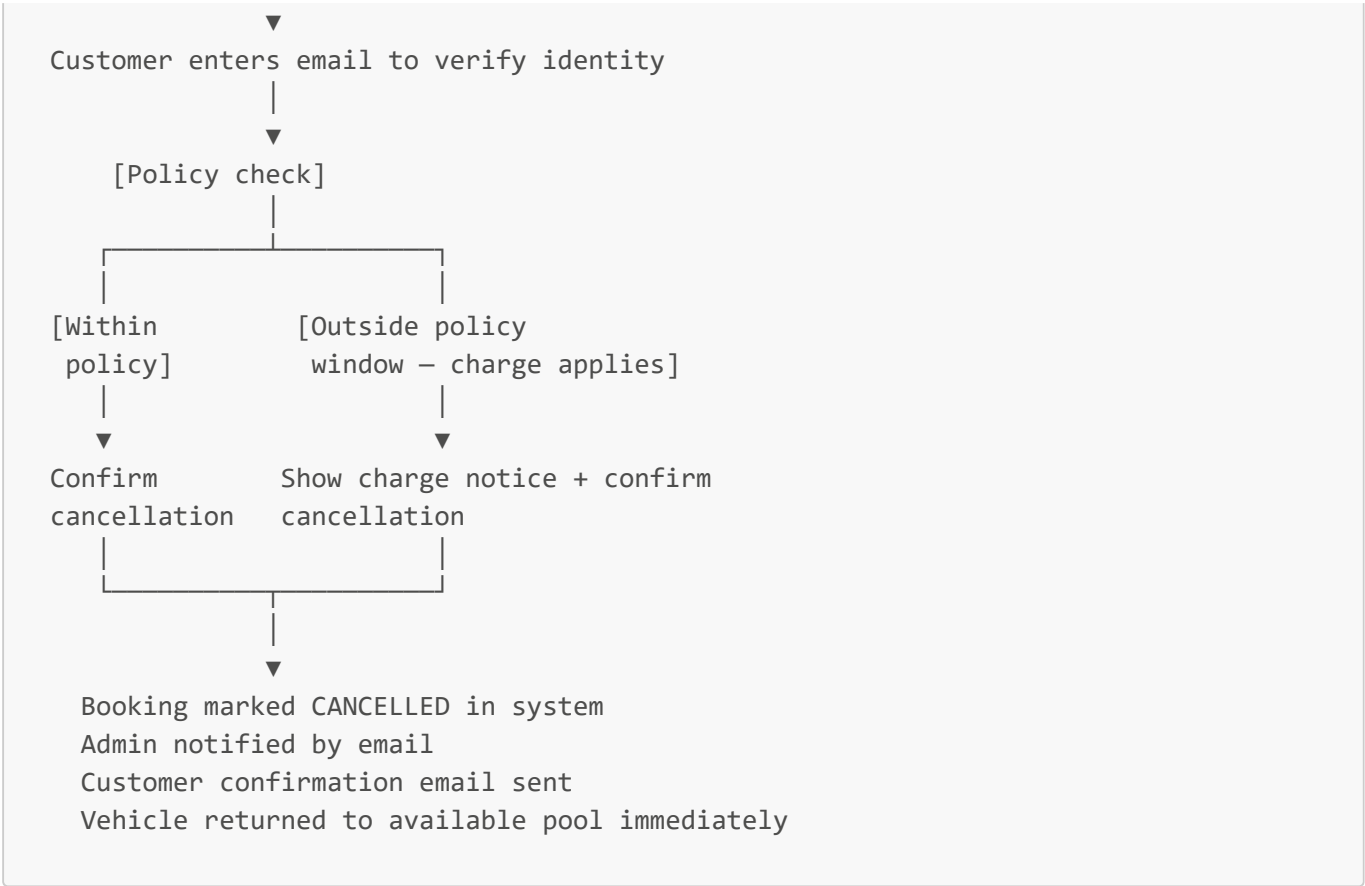
Time before pickup	Cancellation outcome
> 48 hours	Full refund via Stripe (3–5 business days to card)
24–48 hours	50% refund via Stripe
< 24 hours	No refund

Current behaviour (Stripe): When a customer cancels, staff processes the refund manually via the Stripe Dashboard (Payments → find the charge → Refund). The amount is returned to the customer's original card within 3–5 business days. Stripe sends an automatic refund receipt to the customer's email.

Future automation: The Stripe Refund API can be called automatically from the admin dashboard based on policy rules — this is a Phase 3 enhancement.

5.2 Cancellation Flow





6. Accessibility & Inclusive Design

6.1 WCAG 2.1 Level AA Compliance

The application must meet **WCAG 2.1 Level AA** as aligned with:

- *Disability Discrimination Act 1992 (Cth)*
- *Disability Inclusion Act 2014 (NSW)*
- Australian Government Digital Service Standard (Criteria 9: Accessible and Inclusive)

Principle	Requirement	Implementation
Perceivable	Text alternatives for images	alt text on all vehicle photos and icons
Perceivable	Captions for video (if used)	Auto-captions minimum
Perceivable	Colour contrast ≥ 4.5:1	Validated with colour contrast analyser
Perceivable	No information by colour alone	Icons + text labels alongside colour coding
Operable	Keyboard accessible	All interactive elements reachable by Tab key
Operable	No keyboard trap	Escape closes modals; focus returns to trigger
Operable	Skip navigation link	"Skip to main content" as first focusable element
Operable	No seizure-inducing animations	No flashing content
Understandable	Language attribute set	<html lang="en-AU">

Principle	Requirement	Implementation
Understandable	Error identification	Inline errors + ARIA associations
Understandable	Labels for all inputs	<label for="..."> on every form field
Robust	Valid HTML	Validated; no duplicate IDs; semantic landmarks
Robust	ARIA roles used correctly	ARIA only where native HTML semantics insufficient

6.2 Alternative Booking Method

A **phone booking option** must be available for customers who cannot use the online form, in compliance with accessibility obligations. The phone number must be:

- Prominently displayed on the landing page
- Visible in the page header on all booking steps
- Included in all email communications

6.3 Colour and Typography

Element	Specification
Body font	Minimum 16px; sans-serif (e.g., Inter, Roboto)
Heading font	Clear hierarchy: H1 > H2 > H3
Primary action colour	High contrast against white background (contrast ≥ 4.5:1)
Error colour	Red (#D32F2F or equivalent) + icon + text — never colour alone
Focus indicator	Visible focus ring on all interactive elements (min 3px offset)
Line height	1.5 for body text; 1.2 for headings

7. Screen-by-Screen UI Guide

7.1 Landing Page Layout

HEADER: [Facility Logo] [Phone: (02) XXXX XXXX]

Special Need Vehicle Rental
Safe, accessible transport for our community

[Check Availability – Primary CTA Button]
Or call us: (02) XXXX XXXX (Mon–Sat 8am–6pm)

[Vehicle Photo]
Sedan

[Vehicle Photo]
Wheelchair Van

[Vehicle Photo]
7-Seater Wagon

- ✓ Trusted care-sector operator
- ✓ Accessible fleet – ramps, anchor points available
- ✓ Community rates – AUD from \$22/hr
- ✓ Easy online booking – 5 minutes

FOOTER: Privacy Policy | Terms & Conditions | Contact
Complaints: NSW Fair Trading | ABN: XX XXX XXX

7.2 Availability Search Panel

Find a vehicle

Pick-up date & time: [06/06/2026 ▼] [09:00 AM ▼]

Return date & time: [07/06/2026 ▼] [09:00 AM ▼]

Vehicle type: [All Types ▼]

i All times shown in Sydney time (AEST/AEDT)

[Search Available Vehicles – Button]

7.3 Vehicle Card (Results List)

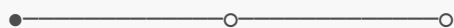
[Photo 200x150] Toyota HiAce – Wheelchair Accessible
 Rear ramp • 4 seats • 2 anchor pts

1 day: AUD \$132.00 (incl. GST)

Hourly: AUD \$22.00/hr

[Book This Vehicle – Button]

7.4 Booking Progress Indicator



Step 1: Details Step 2: Terms Step 3: Payment

Steps are visually indicated; completed steps show a checkmark.

7.5 Step 3 — Stripe Payment Screen Layout

Step 3 of 3 – Payment

Total charge (incl. GST)AUD \$165.00

GST included:\$15.00

Stripe Payment Element (hosted by Stripe)

CardBank[Other methods if available]

Card number: [1234 5678 9012 3456]

Expiry: [MM / YY]CVC: []

 Secured by Stripe. Your card details are never stored on our servers.

[← Back]

[Pay \$165.00 AUD]

The Stripe Payment Element automatically shows Apple Pay or Google Pay buttons above the card form if the user's browser and device support them (e.g. Safari on iPhone, Chrome on Android).

8. Mobile & Responsive Design

The application must be fully usable on mobile devices, as many elderly care family members and carers primarily use smartphones.

Breakpoint	Behaviour
< 640px (mobile)	Single-column layout; full-width buttons; larger tap targets (min 44×44px)
640–1024px (tablet)	2-column layout for vehicle cards
> 1024px (desktop)	Full multi-column layout; sidebar filters

- Touch targets: minimum 44×44 CSS pixels (WCAG 2.5.5)
- Date/time pickers: optimised for mobile (native date pickers + manual text entry fallback)
- Forms: auto-zoom prevention — font-size minimum 16px on input fields
- Images: responsive (**srcset**, WebP format with JPEG fallback)
- Booking summary: sticky sidebar on desktop; collapsible accordion on mobile

9. Email Notification Flows

9.1 Customer Emails

Trigger	Email Sent To	Subject
Booking + payment confirmed (Stripe success)	Customer	"Booking confirmed — SNVR-[ID]"
Stripe payment receipt	Customer	Automatic Stripe receipt from stripe.com
24 hours before pickup	Customer	"Reminder: Your vehicle hire tomorrow"
Vehicle picked up	Customer	"Enjoy your trip — return by [Time]"
Booking completed	Customer	"Thank you — booking complete. Share your feedback."
Booking cancelled	Customer	"Your booking has been cancelled — SNVR-[ID]"
Stripe refund issued	Customer	Automatic Stripe refund receipt from stripe.com

9.2 Admin/Staff Emails

Trigger	Email Sent To	Action Required
New booking submitted (payment confirmed)	Staff inbox	Prepare vehicle for pickup — payment already received
Booking cancelled by customer	Staff inbox	Issue Stripe refund via Stripe Dashboard if applicable
Vehicle maintenance due	Staff inbox	Schedule service; update system

9.3 Confirmation Email Content (Customer)

Subject: Your booking request – SNVR-20260615-042

Dear Jane Smith,

Thank you for choosing Special Need Vehicle Rental.

BOOKING DETAILS

Booking Reference: SNVR-20260615-042
Vehicle: Toyota HiAce (Wheelchair Accessible)
Pick-up: Sunday 15 June 2026, 9:00 AM AEST
Return: Monday 16 June 2026, 9:00 AM AEST
Total: AUD \$132.00 (incl. GST)

PAYMENT CONFIRMED

Your payment of AUD \$132.00 has been received via Stripe.

Your booking is CONFIRMED. A separate receipt from Stripe will be emailed to you shortly.

PICKUP LOCATION
[Facility Name]
[Street Address]
[Suburb, NSW, Postcode]

If you need to cancel: [Cancellation Link]
Questions? Call us: (02) XXXX XXXX
Email: rentals@facility.com.au

Privacy Policy: [Link] | Terms & Conditions: [Link]

This email was sent to jane.smith@example.com
[Facility Name] ABN: XX XXX XXX

10. Cloud Hosting Cost Estimate

Target: ≤ **AUD \$35 / month total**

10.1 Comparison of Recommended Options

	Option A: DigitalOcean	Option B: AWS Lightsail	Option C: AWS EC2
Instance	Basic Droplet — 1 vCPU, 1 GB RAM, 25 GB SSD	1 vCPU, 1 GB RAM, 40 GB SSD	t4g.nano — 2 vCPU, 512 MB RAM, 20 GB EBS
Sydney region	Yes (SYD1)	Yes (ap-southeast-2)	Yes (ap-southeast-2)
Base price (AUD/mo)	~\$10	~\$10	~\$4 (1-year reserved)
Backups	~\$2 (managed snapshots)	~\$3 (manual snapshots)	~\$2 (S3 rsync)
Domain (com.au)	~\$2	~\$2 (Route 53)	~\$2
TLS	Free (Let's Encrypt)	Free (Let's Encrypt)	Free (Let's Encrypt)
Total est. (AUD/mo)	~\$14	~\$15	~\$8–10
Ease of setup	Excellent	Good	Complex
Managed backups	Yes	Partial	Manual
Recommendation	Best for simplicity	Good alternative	Best for lowest cost

All options are well within the AUD \$35/month budget.

10.2 Architecture Notes

- **Database:** SQLite stored on instance disk. For 100 vehicles with low traffic, the database file will be < 10 MB. No managed database (RDS, Firestore) is needed or cost-justified.
- **Cache:** In-memory Node.js cache. No Redis or Memcached needed.
- **Static assets:** React build served directly by Nginx — no S3 or CDN needed at this scale.
- **Email:** Free tier of SMTP provider (Mailgun, SendGrid, or Amazon SES) covers low volume (< 100 emails/day expected). Estimated cost: \$0–\$5/month.
- **Uptime monitoring:** UptimeRobot free tier (50 monitors, 5-minute checks) — \$0/month.
- **SSL:** Let's Encrypt — \$0.

10.3 Cost as Traffic Grows

If bookings grow significantly (> 500/day), consider:

Upgrade	Trigger	Additional Cost
Larger VPS tier	CPU consistently > 70%	+\$5–10/mo
PostgreSQL managed DB	Data > 1 GB or need HA	+\$15–30/mo
CDN (Cloudflare free)	Slow load times internationally	\$0 (free tier)
Load balancer	Multiple instances needed	+\$12/mo (DigitalOcean)

Even at significant growth, costs remain well under AUD \$100/month.

11. Deployment Checklist

Use this checklist when deploying the application for the first time.

11.1 Infrastructure

- ☐ VPS provisioned in Sydney region
- ☐ SSH key authentication configured (password auth disabled)
- ☐ Ubuntu 22.04 LTS fully updated (`apt update && apt upgrade`)
- ☐ Firewall configured: allow 22 (SSH), 80, 443; deny all others
- ☐ Deploy user created (not root) with limited sudo access
- ☐ Node.js 20 LTS installed
- ☐ Nginx installed and enabled
- ☐ Certbot installed

11.2 Application

- ☐ Repository cloned to `/var/www/rental`
- ☐ `.env` file created with production values (no defaults used)
- ☐ Admin password hash generated with bcrypt (not plaintext)
- ☐ `npm install --production` run
- ☐ Database migrations run: `npm run migrate`
- ☐ Fleet seeded or entered manually: `npm run seed`

- ☐ React frontend built: `npm run build` in `/frontend`
- ☐ PM2 configured and started: `pm2 start ecosystem.config.js --env production`
- ☐ PM2 startup hook installed: `pm2 startup && pm2 save`
- ☐ **Stripe:** `STRIPE_SECRET_KEY` set to live key (`sk_live_...`) in `.env`
- ☐ **Stripe:** `VITE_STRIPE_PUBLISHABLE_KEY` set to live key (`pk_live_...`) in `frontend/.env`
- ☐ **Stripe:** Webhook endpoint registered in Stripe Dashboard → Developers → Webhooks
 - URL: `https://rentals.facilitydomain.com.au/api/v1/payments/webhook`
 - Events: `payment_intent.succeeded`, `payment_intent.payment_failed`
- ☐ **Stripe:** `STRIPE_WEBHOOK_SECRET` (starts `whsec_...`) pasted into `.env`
- ☐ **Stripe:** Frontend rebuilt after updating `frontend/.env`: `npm run build`

11.3 Web Server & TLS

- ☐ Nginx site config installed and enabled
- ☐ `nginx -t` passes with no errors
- ☐ Let's Encrypt certificate obtained: `certbot --nginx -d rentals.facilitydomain.com.au`
- ☐ HTTPS redirect confirmed (HTTP → HTTPS)
- ☐ Security headers verified (HSTS, X-Frame-Options, CSP)
- ☐ Certbot auto-renewal tested: `certbot renew --dry-run`

11.4 Backups & Monitoring

- ☐ Daily DB backup cron job configured and tested
- ☐ Backup verified: backup file created, readable, integrity checked
- ☐ UptimeRobot (or similar) configured to monitor `/api/v1/health`
- ☐ Alert email configured for uptime failures
- ☐ PM2 log rotation configured (`pm2 install pm2-logrotate`)

11.5 Pre-Launch Checks

- ☐ Landing page loads correctly in Chrome, Firefox, Safari, Mobile Chrome
- ☐ Availability search returns results
- ☐ End-to-end booking flow completed (test reservation submitted and received)
- ☐ Admin login works; test reservation visible in dashboard
- ☐ Confirmation email received with correct content
- ☐ Cancellation link in email functions correctly
- ☐ Accessibility audit completed (axe browser extension on all key pages)
- ☐ Privacy Policy page accessible
- ☐ Terms & Conditions page accessible
- ☐ Phone number visible and correct on all pages
- ☐ GST-inclusive pricing shown correctly
- ☐ **Stripe test:** Complete a booking using test card `4242 4242 4242 4242` — confirm reservation shows as `paid` in admin
- ☐ **Stripe test:** Complete a booking using test card `4000 0000 0000 0002` — confirm failure page is shown, no reservation confirmed
- ☐ **Stripe live:** After switching to live keys, complete one real \$1.00 test payment and then refund it via Stripe Dashboard before accepting public bookings

12. Operational Launch Checklist

Completed by facility management and operations staff before accepting public bookings.

12.1 Legal & Insurance

- ☐ NSW Fair Trading — motor vehicle hire licensing confirmed (in writing)
- ☐ Insurer has confirmed in writing that the policy covers third-party hire
- ☐ CTP for all 100 vehicles confirmed to cover hire operations
- ☐ Vehicle Hire Agreement drafted and reviewed by solicitor
- ☐ Privacy Policy reviewed by solicitor or privacy adviser
- ☐ Terms & Conditions reviewed and signed off
- ☐ Cancellation policy documented and displayed on site

12.2 Fleet Readiness

- ☐ All vehicles entered in system with photos, rates, and accessibility notes
- ☐ All vehicles have current NSW registration
- ☐ All vehicles serviced and roadworthy
- ☐ Accessible vehicle ramps, lifts, and anchor points inspected and operational
- ☐ Vehicle maintenance schedule established in system
- ☐ Pre-hire checklist printed and available at reception

12.3 Operations Readiness

- ☐ Staff trained on booking system (admin dashboard)
- ☐ Staff trained on hire agreement and key handover procedure
- ☐ Staff trained on privacy obligations
- ☐ Key cabinet installed and keys tagged
- ☐ Damage reporting procedure documented and communicated to staff
- ☐ Incident response procedure posted at reception
- ☐ Complaints procedure established and displayed on website

12.4 Marketing

- ☐ "Special Need Vehicle Rental" link added to facility website (by web team)
- ☐ Email announcement drafted for facility families
- ☐ Launch email sent to existing resident families and community contacts
- ☐ Printed flyers placed at reception, common areas, and car park
- ☐ Staff briefed to mention the service to relevant enquiries
- ☐ Google My Business listing updated (optional)

13. Payment Testing Guide (Stripe)

This section explains how to test the Stripe payment integration end-to-end using test mode. No real money is involved at any point while the test keys are active.

13.1 How Test Mode Works

Stripe provides two sets of API keys:

- **Test keys** (`pk_test_...` / `sk_test_...`) — currently active. Use fake card numbers. No real money moves.
- **Live keys** (`pk_live_...` / `sk_live_...`) — used in production only. Charges real cards.

When test keys are active, the Stripe Payment Element still renders exactly as it will in production — the only difference is that Stripe accepts fake card numbers instead of real ones.

13.2 Test Card Numbers

Use these card numbers in the Stripe Payment Element on the booking page. For all test cards:

- **Expiry date:** Any future date (e.g. `12/28` or `01/30`)
- **CVC:** Any 3 digits (e.g. `123`)
- **Postcode/ZIP:** Any value (e.g. `2000`)

Successful Payments

Card Number	Behaviour	Use to test
4242 4242 4242 4242	Payment succeeds immediately	Standard successful booking
4000 0036 0000 0002	Payment succeeds (Visa Debit)	Debit card scenario
5555 5555 5555 4444	Payment succeeds (Mastercard)	Mastercard scenario
3782 822463 10005	Payment succeeds (Amex)	American Express (4-digit CVC)

Declined / Failed Payments

Card Number	Behaviour	Use to test
4000 0000 0000 0002	Card declined (generic)	Declined card error page
4000 0000 0000 9995	Insufficient funds	Insufficient funds message
4000 0000 0000 0069	Expired card	Expired card message
4000 0000 0000 0127	Incorrect CVC	CVC mismatch message

3D Secure (Additional Authentication)

Some cards require extra verification (a popup from the bank). Stripe simulates this in test mode:

Card Number	Behaviour	Use to test
4000 0025 0000 3155	3D Secure required — customer must authenticate	3DS popup flow
4000 0027 6000 3184	3D Secure — customer fails authentication	Authentication failure

13.3 How to Run a Full Test End-to-End

Step 1 — Start the application Both servers must be running (Backend on port 8080, Frontend on port 5173).

Step 2 — Make a booking

1. Open `http://localhost:5173`
2. Click "Check Availability" and search for dates
3. Select a vehicle and click "Book This Vehicle"
4. Fill in Step 1 (use any name/email/phone — it's all test data)
5. Agree to Terms in Step 2 and click "Next: Payment"
6. Wait for the Stripe Payment Element to load (1–3 seconds)
7. Enter the test card number `4242 4242 4242 4242`, any future expiry, any CVC
8. Click "Pay \$X.XX AUD"

Step 3 — Verify payment succeeded

- Browser redirects to the confirmation page showing "Booking Confirmed!" with a green "Payment received" banner
- Payment status on the page shows **Paid**

Step 4 — Verify in admin dashboard

1. Open `http://localhost:5173/admin/login`
2. Username: `admin` / Password: `admin`
3. Go to Reservations — find the booking
4. Confirm `payment_status = paid` and `status = confirmed`

Step 5 — Verify in Stripe Dashboard

1. Log into your Stripe account at `https://dashboard.stripe.com`
2. Make sure you are in **Test mode** (orange toggle, top-left)
3. Go to **Payments** — your test payment should appear
4. Click it to see the PaymentIntent details, amount, and customer metadata

13.4 Testing a Failed Payment

1. Complete Steps 1–6 above
2. Enter the declined card number `4000 0000 0000 0002`, any future expiry, any CVC
3. Click "Pay"
4. The page should show an error: *"Your card was declined."*
5. The reservation in the admin should NOT be confirmed — `payment_status` remains `none` or `failed`

13.5 Testing 3D Secure Authentication

1. Complete Steps 1–6 above
 2. Enter `4000 0025 0000 3155` as the card number
 3. A Stripe popup will appear asking to "Authorise" or "Fail" the payment
 4. Click **"Authorise"** → payment succeeds, confirmation page shows
 5. Repeat and click **"Fail"** → payment fails, failure page shows
-

13.6 Viewing Test Payments in Stripe Dashboard

Location	What you find
Dashboard → Payments	All test PaymentIntents — status, amount, card used
Click a payment → Metadata	Shows <code>reservation_id</code> , <code>customer_name</code> , <code>customer_email</code>
Dashboard → Developers → Webhooks	Webhook event log (once STRIPE_WEBHOOK_SECRET is configured)
Dashboard → Radar	Fraud rules (relevant in production only)

13.7 Switching to Live Keys for Production

When you are ready to accept real payments:

1. In your Stripe Dashboard, go to **Developers** → **API keys**
2. Toggle to **Live mode** (orange indicator disappears — now in green "Live" mode)
3. Copy the **Live publishable key** (`pk_live_...`) → paste into `frontend/.env` as `VITE_STRIPE_PUBLISHABLE_KEY`
4. Copy the **Live secret key** (`sk_live_...`) → paste into `.env` on the production server as `STRIPE_SECRET_KEY`
5. Register a live webhook at Stripe Dashboard → Developers → Webhooks → Add endpoint:
 - URL: `https://rentals.facilitydomain.com.au/api/v1/payments/webhook`
 - Events: select `payment_intent.succeeded` and `payment_intent.payment_failed`
 - Copy the signing secret (`whsec_...`) → paste into `.env` as `STRIPE_WEBHOOK_SECRET`
6. Rebuild the frontend on the production server: `cd frontend && npm run build`
7. Reload the backend: `pm2 reload rental-api`
8. Complete one real test booking with a low amount (e.g. change a vehicle's hourly rate temporarily to \$1.00), verify payment processes, then refund it from the Stripe Dashboard before opening to the public

Important: Never commit live keys to any code repository. Store them only in the `.env` file on the production server. The `.env` file is in `.gitignore` and is never uploaded.

13.8 Processing a Refund (Staff Guide)

When a customer cancels and is entitled to a refund:

1. Log into Stripe Dashboard → **Payments**
2. Search for the customer's email or booking reference
3. Click the payment to open it
4. Click **"Refund"** button (top right)
5. Enter the refund amount (full or partial, as per the cancellation policy)
6. Add a reason note (optional but recommended)
7. Click **"Refund"**

The customer receives an automatic refund confirmation email from Stripe. The amount returns to their card within **5–10 business days** (Stripe's standard timeframe; most Australian banks process within 3–5 business

days).

Also update the reservation in the admin dashboard: set `payment_status` to `refunded`.

This guide is a living document. Update after each deployment, significant change, or operational review.